

Sumário do Notebook: Análise de Dados com Python

Este notebook explora conceitos e aplicações práticas de análise de dados, com base nos Módulos 3 e 4 do curso "Análise de Dados | Meus primeiros passos em Python!".

Módulo 3 - Primeira Análise

- **Uso da Biblioteca Pandas:**

- Carregamento e visualização de dados (`pd.read_excel, dados.head()`).
- Inspeção de colunas (`dados.columns`).
- Filtragem de dados com base em condições simples e múltiplas (`dados[dados['GENERO'] == 'Feminino'], dados[(condição1) & (condição2)]`).
- Seleção de colunas específicas após a filtragem.
- Agrupamento e contagem de valores únicos (`groupby().nunique()`).
- Contagem de frequência de valores (`value_counts()`), incluindo valores nulos (`dropna=False`).
- Uso de tabelas dinâmicas (`pd.pivot_table`) para análises cruzadas.
- Carregamento de dados de licitações (`licitacoes.xlsx`) e inspeção inicial.

Módulo 4 - Análises Estatísticas com Python

- **4.1 - Estatística Básica:**

- Cálculo de medidas de tendência central (média, mediana, moda) e dispersão (desvio padrão) para listas e colunas de DataFrames.
- Determinação de valores mínimos e máximos.
- Cálculo de estatísticas segmentadas (e.g., média de idade e salário por gênero).
- Análise de diferença salarial entre gêneros.

- **4.2 - Tratamento de Valores Faltantes:**

- Identificação de valores nulos (`dados.info(), isnull()`).
- Preenchimento de valores nulos na coluna 'GENERO' com uma categoria específica (`fillna()`).
- Preenchimento de valores nulos na coluna 'IDADE' com a média, segmentado por 'FAIXA IDADE'.
- Preenchimento de valores nulos na coluna 'SALARIO' com a mediana.

- Tratamento de valores faltantes em colunas de data (Data Abertura) na planilha de licitações.
- **4.3 - Tratamento de Valores Discrepantes (Outliers):**
 - Identificação de outliers usando o método do IQR (Intervalo Interquartil) com boxplots.
 - Definição de limites superior e inferior para detecção de outliers.
 - Tratamento de outliers na coluna 'SALARIO', substituindo-os pela média da respectiva faixa salarial.
- **4.4 - Intervalo de Confiança e Distribuição Amostral:**
 - Cálculo de média, desvio padrão e erro padrão da amostra para a coluna 'SALARIO'.
 - Determinação de intervalos de confiança para a média salarial.
- **4.5 - Feature Engineering (Criação de Novas Variáveis):**
 - Criação de novas categorias de 'NIVEL' (NOVO_NIVEL) baseadas na condição de 'GESTOR?'.
 - Classificação de 'IDADE' em 'GERACAO' (Geração Z, Millenial, Geração X).
 - Combinação de DataFrames via merge (pd.merge).
 - Criação de indicadores booleanos (EM_BUSCA, ABERTO_OPORTUNIDADES) a partir de colunas de texto.
 - Categorização de 'COR/RACA/ETNIA' em 'ETNIA' (brancas, não branca, outras).
- **4.6 - Correlação (Dados Discretos e Contínuos):**
 - Cálculo de correlação de Pearson para variáveis contínuas (e.g., 'IDADE' e 'SALARIO').
 - Implementação e uso do coeficiente de Cramer's V para medir a associação entre variáveis categóricas (e.g., 'COR/RACA/ETNIA' vs 'NIVEL DE ENSINO', 'NIVEL DE ENSINO' vs 'GENERO').
 - Criação de tabelas de contingência (pd.crosstab) para visualização da relação entre variáveis categóricas.

Índice de Importações⁵

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import chi2_contingency
```

Índice de Comandos para Consulta Rápida

Este índice é um guia rápido para os principais comandos e funções utilizados neste notebook, organizados por biblioteca e funcionalidade.

1. Biblioteca Pandas (pd)

1.1 Carregamento e Visualização de Dados

- `pd.read_excel('/caminho/do/arquivo.xlsx')`: Carrega dados de um arquivo Excel para um DataFrame.
- `pd.read_csv('/caminho/do/arquivo.csv', encoding='latin1', sep=';')`: Carrega dados de um arquivo CSV, especificando a codificação e o separador.
- `dados.head()`: Exibe as primeiras 5 linhas do DataFrame.
- `dados.columns`: Lista os nomes de todas as colunas do DataFrame.
- `dados.info()`: Fornece um resumo conciso do DataFrame, incluindo tipo de dados e contagem de valores não nulos.
- `display(df)`: Exibe o DataFrame completo (ou parte dele, dependendo da configuração).

1.2 Filtragem e Seleção de Dados

- `dados['COLUNA']`: Seleciona uma única coluna do DataFrame.
- `dados[dados['COLUNA'] == 'VALOR']`: Filtra linhas onde 'COLUNA' tem um 'VALOR' específico.
- `dados[(condição1) & (condição2)]`: Filtra linhas com múltiplas condições (use `|` para 'OU').
- `dados[['COLUNA1', 'COLUNA2']]`: Seleciona múltiplas colunas.

1.3 Agrupamento e Contagem

- `dados.groupby('COLUNA')['ID'].nunique()`: Agrupa por 'COLUNA' e conta o número de IDs únicos em cada grupo.
- `dados.groupby('COLUNA', dropna=False)['ID'].nunique()`: Inclui valores nulos no agrupamento para contagem de IDs únicos.
- `dados['COLUNA'].value_counts(dropna=False)`: Conta a frequência de valores únicos em uma coluna, incluindo nulos.
- `pd.pivot_table(dados, values=['VALOR'], index=['INDICE'], columns=['COLUNAS'], aggfunc='count')`: Cria uma tabela dinâmica para análise cruzada.

1.4 Tratamento de Valores Faltantes (NaN)

- `dados['COLUNA'].isnull().sum()`: Conta o número de valores nulos em uma coluna.
- `dados['COLUNA'].fillna('VALOR_PADRAO')`: Preenche valores nulos em 'COLUNA' com um 'VALOR_PADRAO'.
- `dados.loc[condição, 'COLUNA'] = NOVO_VALOR`: Atribui 'NOVO_VALOR' a linhas que satisfazem a 'condição' na 'COLUNA' específica.
- `pd.to_datetime(df, errors='coerce')`: Converte uma coluna para o tipo datetime, transformando erros em NaT.

1.5 Feature Engineering

- `dados.apply(lambda x: minha_funcao(x['COLUNA1'], x['COLUNA2']), axis=1)`: Aplica uma função a cada linha do DataFrame para criar uma nova coluna.
- `pd.get_dummies(dados, columns=['COLUNA'])`: Converte colunas categóricas em variáveis dummy (one-hot encoding).
- `dados.merge(outro_df, on='ID', how='left')`: Combina dois DataFrames com base em uma coluna comum.
- `dados['COLUNA'].str.contains('texto', case=False)`: Cria uma coluna booleana baseada na presença de um 'texto' (ignorando maiúsculas/minúsculas).

2. Biblioteca NumPy (np)

2.1 Funções Matemáticas e Estatísticas Básicas

- `np.sum(lista)`: Calcula a soma dos elementos de uma lista ou array.
- `len(lista)`: Retorna o número de elementos em uma lista.
- `np.mean(lista)`: Calcula a média aritmética dos elementos.
- `np.median(lista)`: Calcula a mediana dos elementos.
- `np.std(lista)`: Calcula o desvio padrão dos elementos.

3. Biblioteca Matplotlib (plt)

3.1 Visualização

- `plt.boxplot(dados['COLUNA'])`: Gera um boxplot para visualizar a distribuição e outliers de uma coluna numérica.
- `plt.show()`: Exibe o(s) gráfico(s) gerado(s).

4. Biblioteca SciPy (stats)

4.1 Estatística Avançada

- `stats.sem(salario)`: Calcula o erro padrão da média de uma amostra.
- `stats.t.interval(nivel_confianca, tamanho_amostra-1, loc=media_amostrai, scale=erro_padrao)`: Calcula o intervalo de confiança para a média usando a distribuição t de Student.
- `chi2_contingency(tabela_cruzada)`: Realiza o teste qui-quadrado de independência em uma tabela de contingência.

5. Funções Personalizadas

5.1 Cramer's V para Correlação Categórica

- `cramer_coeficiente(coluna1, coluna2)`: Função customizada para calcular o coeficiente V de Cramer, que mede a associação entre duas variáveis categóricas.
 - `np.array(pd.crosstab(coluna1, coluna2))`: Cria uma tabela de contingência a partir de duas colunas categóricas.
 - `np.sqrt(chi2/(soma*mini))`: Fórmula do V de Cramer, utilizando o valor do qui-quadrado (chi2), a soma total das observações (soma) e o mínimo de (linhas-1, colunas-1) (mini).

